

JavaScript RegExp Flags

[Back to JS page](#)

Table of Contents

- [JavaScript RegExp Flags](#)
 - [RegExp /g Flag \(Global\)](#)
 - [Example 1](#)
 - [RegExp /i Flag \(Insensitive\)](#)
 - [Example 2](#)
 - [RegExp /m Flag \(Multiline\)](#)
 - [Example 3](#)
 - [RegExp /y Flag \(Sticky\)](#)
 - [Example 4](#)
 - [RegExp /u Flag \(Unicode\)](#)
 - [Example 5](#)
 - [Document](#)
 - [Reference](#)

RegExp Modifier Flags

Flags modify how a regular expression searches for patterns. The syntax is `/pattern/flags` :

Flag	Description
<code>/g</code>	Global - matches all occurrences
<code>/i</code>	Insensitive - case insensitive
<code>/m</code>	Multiline - <code>^</code> and <code>\$</code> match line breaks
<code>/s</code>	DotAll - <code>.</code> matches newlines too
<code>/y</code>	Sticky - matches only from <code>lastIndex</code>
<code>/u</code>	Unicode - proper Unicode handling

RegExp /g Flag (Global)

The `/g` flag searches for **all** occurrences, not just the first:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Flags</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>RegExp /g Flag (Global)</h4>
<p id="demo"></p>

<script>
const text = "apple banana apple cherry apple";

// Without /g - only first match
const first = text.match(/apple/);

// With /g - all matches
const all = text.match(/apple/g);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "without /g: " + first + "<br>" +
  "with /g: " + all + " (all occurrences)";
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

RegExp /i Flag (Insensitive)

The `/i` flag makes the search **case insensitive**:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Flags</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>RegExp /i Flag (Insensitive)</h4>
<p id="demo"></p>

<script>
const text = "JavaScript is FUN!";

// Case sensitive
const sensitive = text.match(/fun/);

// Case insensitive
const insensitive = text.match(/fun/i);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "match(/fun/): " + sensitive + " (no match)<br>" +
  "match(/fun/i): " + insensitive + " (matched)";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

RegExp /m Flag (Multiline)

The `/m` flag makes `^` and `$` match the beginning/end of each line:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Flags</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>RegExp /m Flag (Multiline)</h4>
<p id="demo"></p>

<script>
const text = "line1\nline2\nline3";

// Without /m - ^ only matches start of string
const withoutM = text.match(/^line/g);

// With /m - ^ matches start of each line
const withM = text.match(/^line/gm);

document.getElementById("demo").innerHTML =
  "Text:<br>" + text + "<br><br>" +
  "match(/^line/g): " + withoutM + "<br>" +
  "match(/^line/gm): " + withM + " (each line)";
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

RegExp /y Flag (Sticky)

The `/y` flag makes the search **sticky** - it only matches starting at `lastIndex` :

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Flags</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>RegExp /y Flag (Sticky)</h4>
<p id="demo"></p>

<script>
const text = "abcabc";
const sticky = /abc/y;

// First match at position 0
const first = sticky.exec(text);

// After lastIndex = 3, matches "abc" again at position 3
const second = sticky.exec(text);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "sticky = /abc/y<br>" +
  "First exec: " + first[0] + " at index " + first.index + "<br>" +
  "lastIndex after: " + sticky.lastIndex + "<br>" +
  "Second exec: " + second[0] + " at index " + second.index;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

RegExp /u Flag (Unicode)

The `/u` flag enables proper Unicode handling:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript RegExp Flags</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>RegExp /u Flag (Unicode)</h4>
<p id="demo"></p>

<script>
const text = "Café 100";

// \d finds ASCII digits only
const asciiDigits = text.match(/\d/g);

document.getElementById("demo").innerHTML =
  "Text: " + text + "<br><br>" +
  "match(/\d/g): " + asciiDigits + "<br><br>" +
  "<b>Other flag properties:</b><br>" +
  "global: " + /a/g.global + "<br>" +
  "ignoreCase: " + /a/i.ignoreCase + "<br>" +
  "multiline: " + /a/m.multiline + "<br>" +
  "sticky: " + /a/y.sticky + "<br>" +
  "unicode: " + /a/u.unicode;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript RegExp Flags](#)